

Quantum Computing + HPC Roadmap From Scratch

This roadmap is designed for a college freshman starting with **no experience** who wants to become competitive for roles like:

- Quantum software engineer
- Quantum simulator engineer
- Quantum-HPC software engineer
- Quantum error-correction decoding engineer
- Quantum runtime / quantum-centric supercomputing engineer
- GPU/distributed quantum simulation engineer
- Scientific computing engineer working on quantum workloads

The goal is not just to “learn quantum computing.” The goal is to become someone who can:

Build, optimize, benchmark, parallelize, and scale classical software for quantum computing workloads.

You want to become the kind of person who understands both:

1. **Quantum computing theory**
 2. **High-performance classical computing used to simulate, compile, decode, and run quantum workloads**
-

Roadmap overview

	Stage	Timeline	Main theme	Main project
	0	Weeks 0–2	Setup and learning habits	Quantum-HPC learning repo
	1	Months 0–3	Programming + math foundations	Complex linear algebra toolkit
	2	Months 3–6	Scientific Python + numerical computing	NumPy state-vector simulator
	3	Months 6–9	Quantum computing fundamentals	Quantum algorithms from scratch
	4	Months 9–12	Algorithms + numerical methods	QFT / Hamiltonian simulation notebooks
	5	Months 12–15	C++ + systems + performance	Serial C++ quantum simulator

Stage	Timeline	Main theme	Main project
6	Months 15–18	OpenMP, MPI, Slurm, HPC workflow	Parallel state-vector simulator
7	Months 18–21	GPU programming + accelerated simulation	CUDA quantum gate kernels
8	Months 21–24	Advanced quantum simulation methods	Simulator benchmark suite
9	Months 24–27	Quantum error correction + decoding	Surface-code decoder benchmark
10	Months 27–30	Capstone + open source + applications	Flagship quantum-HPC project

Stage 0 — Weeks 0–2: setup and learning habits

Goal

Build the technical environment you will use for the next two years.

Learn

You should become comfortable with:

- Terminal / command line
- Git and GitHub
- VS Code or another editor
- Python environments
- Jupyter notebooks
- Basic Linux commands
- Installing packages
- Running tests
- Writing README files
- Reproducible project folders

Updated textbooks and resources for this stage

Core

Dive Into Systems — free

Use this as your first systems/programming textbook. It introduces C, memory, computer organization, and systems concepts.

Think Python by Allen Downey — free

Use this if Python is completely new to you. It is beginner-friendly and helps you build basic programming fluency.

The Pragmatic Programmer by Andrew Hunt and David Thomas — paid, optional early

Do not read the whole thing yet. Use it to learn good habits: documentation, debugging, project organization, and clean thinking.

Supporting resources

- Harvard CS50x
- Software Carpentry Unix Shell
- Software Carpentry Git
- GitHub Docs
- Python official documentation

Install

Set up:

- Git
- GitHub account
- VS Code
- Python 3.11+
- uv, conda, or venv
- JupyterLab
- NumPy
- SciPy
- Matplotlib
- pytest
- Qiskit
- Qiskit Aer
- Cirq
- PennyLane
- Qulacs
- Stim
- PyMatching
- C/C++ compiler

- CMake
- OpenMP
- MPI implementation later: OpenMPI or MPICH
- CUDA later, only if you have an NVIDIA GPU

Project 0: quantum-hpc-learning-log

Create a public GitHub repo.

Structure:

```
quantum-hpc-learning-log/  
  week_01/  
    notes.md  
    exercises/  
    code/  
  week_02/  
    resources.md  
    roadmap.md
```

Every week, add:

- What you studied
- Problems solved
- Code written
- One concept explained in your own words
- One plot, benchmark, or small demo when possible

Move on when

You can:

- Clone a repo
 - Make commits
 - Create a Python environment
 - Run a Jupyter notebook
 - Install Python packages
 - Run a simple Python script from the terminal
 - Write a basic README
-

Stage 1 — Months 0–3: programming and math foundations

This stage is not “quantum” yet, but it is essential. Quantum computing is mostly **linear algebra, probability, complex numbers, and algorithms**. HPC is mostly **programming, memory, performance, and parallelism**.

Topics to learn

Programming

- Python variables, functions, loops, lists, dictionaries
- File I/O
- Debugging
- Basic testing with pytest
- Basic C from CS50
- Basic data structures
- Git/GitHub
- Command line
- How to read documentation

Math

- Algebra review
- Trigonometry
- Complex numbers
- Complex conjugates
- Polar form
- Euler's formula
- Vectors
- Matrices
- Matrix multiplication
- Dot products
- Norms
- Matrix-vector multiplication
- Basic probability
- Basic calculus

Updated textbooks and resources for this stage

Core textbooks

Think Python by Allen Downey — free

Use for beginner Python if you are starting from zero.

Dive Into Systems — free

Use for C basics, memory, pointers, and computer-systems intuition.

Introduction to Linear Algebra by Gilbert Strang — paid

Use as your main linear algebra textbook. It is especially good for building intuition around vectors, matrices, eigenvalues, and orthogonality.

Open Data Structures — free

Use for your first exposure to arrays, lists, stacks, queues, hash tables, trees, and graphs.

Optional / deeper textbooks

C Programming: A Modern Approach by K. N. King — paid

Use if you want a thorough C book.

Computer Science: An Interdisciplinary Approach by Sedgewick and Wayne — paid

Use if you want a programming + scientific computing style introduction.

Introduction to Probability by Blitzstein and Hwang — paid

Use when you start probability. This will be useful later for quantum measurement, Monte Carlo simulation, QEC, and randomized algorithms.

Supporting resources

- Harvard CS50x
- MIT 18.06 Linear Algebra
- 3Blue1Brown Essence of Linear Algebra
- Khan Academy complex numbers
- MIT 18.05 Probability and Statistics

Project 1: complex linear algebra toolkit

Build a Python repo called:

complex-linear-algebra-lab

Implement from scratch first, then compare to NumPy.

Functions:

- Complex addition/multiplication helper functions
- Vector addition
- Scalar-vector multiplication
- Dot product
- Complex inner product
- Norm
- Matrix-vector multiplication
- Matrix-matrix multiplication
- Identity matrix
- Conjugate transpose
- Check if a matrix is unitary
- Check if a matrix is Hermitian

Add:

- Unit tests
- Example notebooks
- Plots of complex numbers on the complex plane

- README explaining each concept

Project 2: qubit probability calculator

Build a small Python project that:

- Represents a qubit as $[\alpha, \beta]$
- Normalizes the state
- Checks that $|\alpha|^2 + |\beta|^2 = 1$
- Computes probability of measuring 0 or 1
- Applies X, Y, Z, H gates as matrices
- Verifies that gates preserve norm

Example README section:

A qubit is represented by a normalized complex vector:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

The probability of measuring 0 is $|\alpha|^2$.

The probability of measuring 1 is $|\beta|^2$.

Move on when

You can:

- Write Python functions without copying every line
 - Explain complex numbers and complex conjugates
 - Multiply matrices by hand for small examples
 - Explain what an eigenvalue is
 - Explain what a unitary matrix is
 - Write tests for your code
 - Use GitHub comfortably
 - Implement single-qubit gates as matrices
-

Stage 2 — Months 3–6: scientific Python and numerical computing

Now you begin learning the tools used in real scientific and quantum software.

Topics to learn

Scientific Python

- NumPy arrays
- Vectorization
- Broadcasting

- Avoiding slow Python loops
- SciPy linear algebra
- SciPy sparse matrices
- Matplotlib plotting
- Random sampling
- Benchmarking
- Profiling
- Floating-point arithmetic
- Numerical error
- Sparse vs dense matrices

Numerical computing

- Dense matrix-vector multiplication
- Sparse matrix-vector multiplication
- Eigenvalue problems
- Matrix exponentials
- Numerical stability
- Conditioning
- Runtime scaling
- Memory scaling

This matters because quantum simulation is brutally memory-intensive. A state-vector simulator stores 2^n complex amplitudes for n qubits. With complex double precision, that is roughly $16 * 2^n$ bytes.

Qubits	State-vector memory
20	~16 MB
25	~512 MB
30	~16 GB
35	~512 GB
40	~16 TB

That exponential scaling is one of the central reasons HPC matters in quantum computing.

Updated textbooks and resources for this stage

Core textbooks

Scientific Python Lectures — free

Use for NumPy, SciPy, Matplotlib, sparse arrays, optimization, debugging, and scientific workflow.

Introduction to Linear Algebra by Gilbert Strang — paid

Continue using it for vector spaces, orthogonality, eigenvalues, and matrix factorization intuition.

Numerical Linear Algebra by Trefethen and Bau — paid

Use once you are comfortable with basic linear algebra. This is one of the most important books for quantum-HPC because scalable quantum simulation is mostly numerical linear algebra.

Scientific Computing: An Introductory Survey by Michael T. Heath — paid

Use for floating-point arithmetic, numerical linear algebra, interpolation, integration, ODEs, and optimization.

Optional / reference textbooks

Matrix Computations by Golub and Van Loan — paid reference

Use later as a deeper numerical linear algebra reference.

Numerical Methods in Engineering with Python by Jaan Kiusalaas — paid

Use if you want practical Python numerical methods examples.

Introduction to Probability by Blitzstein and Hwang — paid

Use for probability foundations, random variables, sampling, expectation, variance, and Monte Carlo thinking.

Supporting resources

- NumPy official tutorials
- SciPy documentation
- MIT 18.335 Numerical Methods
- MIT 18.05 Probability and Statistics

Project 3: NumPy state-vector simulator v0

Build:

numpy-quantum-simulator

Features:

- Initialize $|0\dots0\rangle$
- Apply single-qubit gates
- Apply CNOT
- Apply arbitrary 2x2 unitary gate
- Measure one qubit
- Measure all qubits
- Sample measurement outcomes
- Build Bell states
- Build GHZ states
- Run small random circuits
- Benchmark runtime vs number of qubits

Important implementation detail:

Do **not** build the full $2^n \times 2^n$ gate matrix for every operation. That is too slow and too memory-heavy. Apply gates directly to the state vector by updating pairs or blocks of amplitudes.

Your README should include:

- Explanation of state-vector simulation
- Memory scaling table
- Runtime scaling plot
- Correctness tests
- Example circuits

Project 4: dense vs sparse Hamiltonian lab

Build notebooks that:

- Construct Pauli matrices
- Construct tensor products
- Build small Hamiltonians
- Compare dense and sparse storage
- Compute eigenvalues
- Use sparse eigensolvers
- Plot runtime and memory usage

This prepares you for Hamiltonian simulation, VQE, quantum chemistry, and tensor-network methods.

Move on when

You can:

- Use NumPy arrays naturally
 - Explain broadcasting
 - Explain why state-vector simulation scales exponentially
 - Apply gates without building giant full matrices
 - Benchmark code
 - Plot runtime vs qubit count
 - Use SciPy sparse matrices
 - Explain dense vs sparse tradeoffs
-

Stage 3 — Months 6–9: quantum computing fundamentals

Now you formally learn quantum computing.

Topics to learn

Quantum basics

- Bits vs qubits
- Computational basis
- Superposition
- Measurement
- Born rule
- Global phase
- Relative phase
- Single-qubit gates
- Multi-qubit gates
- Tensor products
- Entanglement
- Bell states
- No-cloning theorem
- Reversible computing
- Universal gate sets
- Circuit diagrams

Basic algorithms and protocols

- Quantum random number generation
- Deutsch-Jozsa
- Bernstein-Vazirani
- Simon's algorithm conceptually
- Grover search
- Quantum teleportation
- Superdense coding
- Quantum Fourier Transform
- Phase estimation intro
- Shor's algorithm conceptually

Updated textbooks and resources for this stage

Core textbooks

Quantum Computing for the Quantum Curious by Ciaran Hughes, Joshua Isaacson, Anastasia Perry, Ranbel F. Sun, and Jessica Turner — free

Use this as your first quantum computing book. It is beginner-friendly and good before more mathematical texts.

Quantum Computer Science: An Introduction by N. David Mermin — paid

Use as your first paid quantum computing textbook. It is clearer and more approachable than Nielsen & Chuang for many beginners.

Quantum Computation and Quantum Information by Nielsen and Chuang — paid
Use as your long-term standard reference. Do not try to read it cover to cover yet. Use the early chapters for qubits, gates, circuits, measurement, entanglement, and basic algorithms.

Quantum Computing: Lecture Notes by Ronald de Wolf — free
Use for a more rigorous algorithms-focused treatment. Start with early chapters after you understand qubits and circuits.

Optional textbooks

Quantum Computing: A Gentle Introduction by Rieffel and Polak — paid
Good alternative or supplement to Mermin.

Introduction to Classical and Quantum Computing by Thomas Wong — free
Good beginner-friendly bridge between classical and quantum computation.

Dancing with Qubits by Robert S. Sutor — paid
Useful if you want a more applied quantum-programming perspective.

Supporting resources

- Quantum Country
- IBM Quantum Learning
- PennyLane Codebook
- Microsoft Quantum Katas
- Qiskit tutorials
- Cirq documentation

Project 5: quantum algorithms from scratch

Extend your simulator to include:

- Deutsch-Jozsa
- Bernstein-Vazirani
- Grover search
- QFT
- Phase estimation for a simple unitary
- Teleportation
- Superdense coding

For each algorithm:

- Explain the problem
- Explain the quantum circuit
- Show state evolution for small inputs
- Run your simulator
- Compare with Qiskit, Cirq, or PennyLane
- Include tests

Project 6: Qiskit/Cirq/PennyLane comparison

Build a repo:

quantum-framework-comparison

Compare:

- Qiskit
- Cirq
- PennyLane
- Your simulator

For each:

- Build the same Bell circuit
- Build GHZ
- Build Grover
- Build QFT
- Time simulation performance for small qubit counts
- Compare output formats
- Compare ease of circuit construction

Move on when

You can:

- Explain measurement probabilities
 - Explain tensor products
 - Build a Bell state without a tutorial
 - Explain entanglement
 - Implement Grover for a tiny search space
 - Implement QFT for 3–5 qubits
 - Use Qiskit or Cirq without feeling lost
 - Compare framework outputs against your simulator
-

Stage 4 — Months 9–12: algorithms, data structures, and numerical methods

Quantum-HPC roles require strong classical CS. Quantum software is still software.

Topics to learn

Algorithms and data structures

- Big-O notation

- Arrays
- Hash tables
- Stacks and queues
- Trees
- Heaps
- Graphs
- BFS
- DFS
- Dijkstra
- Dynamic programming
- Sorting
- Recursion
- Complexity classes intro

Discrete math

- Proofs
- Induction
- Sets
- Relations
- Modular arithmetic
- Graph theory
- Counting
- Discrete probability

Numerical methods

- Floating-point arithmetic
- Condition number
- Stability
- Matrix factorizations
- QR decomposition
- SVD conceptually
- Eigenvalue algorithms
- Krylov methods conceptually
- Sparse iterative solvers
- FFT
- Optimization basics

Updated textbooks and resources for this stage

Core textbooks

Introduction to Algorithms by Cormen, Leiserson, Rivest, and Stein, usually called CLRS — paid

Use as the standard algorithms reference for data structures, graphs, dynamic programming, sorting, complexity, and algorithmic analysis.

Algorithms by Dasgupta, Papadimitriou, and Vazirani — free

Use as a more readable free algorithms book. This is a strong alternative to CLRS.

Open Data Structures — free

Use for implementation-oriented data structures.

Numerical Linear Algebra by Trefethen and Bau — paid

Use for numerical matrix algorithms, conditioning, stability, and eigenvalue problems.

Scientific Computing: An Introductory Survey by Michael T. Heath — paid

Use for practical numerical methods, floating point, ODEs, optimization, and scientific-computing thinking.

Quantum Computing: Lecture Notes by Ronald de Wolf — free

Use for QFT, Grover, phase estimation, and algorithmic quantum computing.

Optional textbooks

Convex Optimization by Boyd and Vandenberghe — free

Useful later for optimization, variational algorithms, and algorithmic thinking.

Numerical Recipes — paid

Use only as a broad reference. Prioritize Trefethen & Bau and Heath first.

Mathematics for Computer Science by Lehman, Leighton, and Meyer — free

Use for proofs, counting, graph theory, probability, and discrete math.

Supporting resources

- MIT 6.006 Introduction to Algorithms
- MIT 6.042J Mathematics for Computer Science
- MIT 18.335 Numerical Methods

Project 7: QFT vs FFT learning lab

Build a technical notebook comparing:

- Classical DFT
- FFT
- Quantum Fourier Transform
- Circuit depth of QFT
- Approximate QFT
- Output interpretation
- Why QFT is not simply “getting all Fourier coefficients for free”

Deliverables:

- Derivations

- Code
- Circuit diagrams
- Runtime discussion
- README explaining the difference between FFT and QFT

This is a very good project because it shows you understand both **classical numerical algorithms** and **quantum algorithms**.

Project 8: classical baselines for quantum algorithms

Build classical baseline implementations for:

- Search
- MaxCut brute force
- MaxCut heuristic
- Small Hamiltonian diagonalization
- Random circuit simulation
- Sparse Hamiltonian simulation

Your goal is to avoid quantum hype. Good quantum engineers know when classical algorithms are better.

Move on when

You can:

- Analyze Big-O runtime
 - Implement graph algorithms
 - Explain FFT at a high level
 - Explain QFT at a high level
 - Use sparse matrices
 - Use iterative solvers
 - Write classical baselines for quantum projects
 - Solve easy/medium algorithm problems without panic
-

Stage 5 — Months 12–15: C++, systems, and performance engineering

This is where you start becoming serious for HPC roles.

Topics to learn

C++

- C++17 or C++20
- `std::vector`

- `std::complex`
- References
- Pointers
- RAII
- Move semantics conceptually
- Templates basics
- CMake
- Header/source organization
- Unit testing
- Benchmarking
- Compiler flags
- Sanitizers
- Debugging with gdb/lldb

Systems and performance

- Stack vs heap
- CPU caches
- Cache lines
- Memory bandwidth
- SIMD/vectorization
- Branching
- Alignment
- False sharing
- Locality
- Profiling
- Latency vs throughput
- FLOPs vs bytes
- Roofline thinking

Updated textbooks and resources for this stage

Core textbooks

A Tour of C++ by Bjarne Stroustrup — paid

Use as your compact introduction to modern C++.

Programming: Principles and Practice Using C++ by Bjarne Stroustrup — paid

Use if you need a full beginner-friendly C++ book instead of a compact overview.

Effective Modern C++ by Scott Meyers — paid

Use after you can write basic C++. It teaches modern best practices, move semantics, smart pointers, lambdas, auto, and performance-aware C++.

Computer Systems: A Programmer's Perspective by Bryant and O'Hallaron — paid
Use for memory hierarchy, linking, machine-level programming, caches, virtual memory, and systems performance.

Dive Into Systems — free
Use as a free systems-programming companion or substitute.

Optional / advanced textbooks

Optimized C++ by Kurt Guntheroth — paid
Use for practical C++ performance optimization.

Computer Architecture: A Quantitative Approach by Hennessy and Patterson — paid
Use later if you want deep architecture and performance-modeling knowledge.

Numerical Linear Algebra by Trefethen and Bau — paid
Continue using it for matrix algorithms and numerical stability.

Supporting resources

- Stanford CS106L
- MIT 6.172 Performance Engineering
- Compiler Explorer
- CMake documentation
- Google Benchmark
- gdb/lldb documentation

Project 9: serial C++ quantum simulator

Build:

cpp-statevector-simulator

Features:

- CMake build
- Unit tests
- `std::vector<std::complex<double>>` state vector
- Single-qubit gate application
- CNOT
- CZ
- SWAP
- Measurement
- Random circuit generator
- JSON or simple text input circuit format
- Benchmark executable
- Correctness tests against your Python simulator

Benchmark:

- Runtime vs qubit count
- Runtime vs circuit depth
- Memory usage
- Gate throughput
- Comparison between naive and optimized gate application

Important implementation concept:

A single-qubit gate acting on qubit k updates amplitude pairs whose indices differ by bit k .

Your README should explain that.

Project 10: performance optimization report

Take your C++ simulator and produce a report:

- Baseline runtime
- Bottleneck hypothesis
- Profiling results
- Optimization attempted
- New runtime
- Speedup
- What did not work
- What you learned

Examples of optimizations:

- Avoid unnecessary allocations
- Improve memory locality
- Precompute masks
- Change qubit ordering
- Use compiler optimization flags
- Use loop restructuring
- Try single precision vs double precision
- Try gate fusion for adjacent gates

Move on when

You can:

- Compile C++ with CMake
- Debug C++ code
- Write tests
- Explain cache locality
- Profile code
- Build a serial simulator that matches Python/Qiskit results

- Produce honest benchmark plots
 - Explain why a “better algorithm” can beat a “faster language”
-

Stage 6 — Months 15–18: parallel computing and HPC workflow

Now you learn actual HPC.

Topics to learn

Shared-memory parallelism

- Threads
- Race conditions
- OpenMP
- Parallel loops
- Reductions
- Scheduling
- Thread affinity
- False sharing
- Strong scaling

Distributed-memory parallelism

- MPI
- Rank/process model
- Send/receive
- Broadcast
- Scatter/gather
- Allreduce
- Domain decomposition
- Communication overhead
- Weak scaling
- Hybrid MPI + OpenMP

HPC workflow

- SSH
- Environment modules
- Slurm
- Batch scripts
- Job arrays
- Cluster file systems
- Reproducibility
- Containers

- Spack
- Performance logs
- Scaling studies

Updated textbooks and resources for this stage

Core textbooks

The Art of HPC by Victor Eijkhout — free

Use as your main free HPC textbook. It covers MPI, OpenMP, scientific computing, and parallel programming patterns.

An Introduction to Parallel Programming by Peter Pacheco — paid

Use as your main paid first textbook for MPI, OpenMP, and parallel programming foundations.

Parallel Programming in C with MPI and OpenMP by Michael J. Quinn — paid

Use as an alternative or supplement for C-based MPI/OpenMP.

Introduction to High Performance Computing for Scientists and Engineers by Hager and Wellein — paid

Use for performance engineering, memory hierarchy, multicore optimization, and scientific computing performance.

Optional / reference textbooks

Using MPI by Gropp, Lusk, and Skjellum — paid

Use as a deeper MPI reference after you know the basics.

Using OpenMP by Chapman, Jost, and van der Pas — paid

Use as a deeper OpenMP reference.

Computer Systems: A Programmer's Perspective by Bryant and O'Hallaron — paid

Continue using for memory hierarchy and performance.

Supporting resources

- LLNL HPC Tutorials
- OpenMP official documentation
- MPI Tutorial
- UC Berkeley CS267
- CMU 15-418/15-618
- Slurm documentation

Project 11: OpenMP state-vector simulator

Add OpenMP to your C++ simulator.

Parallelize:

- Single-qubit gates

- Two-qubit gates
- Probability calculation
- Measurement sampling helper functions
- Random circuit execution

Benchmark:

- 1 thread
- 2 threads
- 4 threads
- 8 threads
- More if available

Measure:

- Speedup
- Efficiency
- Memory bandwidth behavior
- Cases where parallelism does not help

README should include:

Strong scaling measures how runtime changes as the number of processors increases for a fixed problem size.

Weak scaling measures how runtime changes as both problem size and processor count increase.

Project 12: MPI distributed state-vector simulator

This is more advanced and very résumé-relevant.

Design:

- Split the state vector across MPI ranks
- Each rank owns a chunk of amplitudes
- Gates on local qubits require local updates
- Gates on distributed qubits require communication
- Implement basic communication strategy
- Benchmark communication-heavy vs communication-light gates

Minimum viable version:

- Initialize distributed state
- Apply X/H/Z on local qubits
- Apply gates that require communication
- Compute global norm with MPI_Allreduce

- Verify correctness against serial version

Deliverables:

- Code
- Tests
- Scaling plots
- Explanation of state partitioning
- Explanation of communication bottlenecks

Move on when

You can:

- Write OpenMP code
 - Explain race conditions
 - Write basic MPI programs
 - Run strong and weak scaling experiments
 - Use Slurm batch scripts
 - Explain why distributed quantum simulation is communication-heavy
 - Produce a scaling report that includes failures and bottlenecks
-

Stage 7 — Months 18–21: GPU programming and accelerated quantum simulation

GPU programming is extremely valuable for quantum-HPC roles.

Topics to learn

CUDA basics

- GPU vs CPU architecture
- Threads, blocks, grids
- Warps
- Global memory
- Shared memory
- Registers
- Coalesced memory access
- Occupancy
- Kernel launches
- Host/device memory transfer
- Reductions
- Streams basics
- CUDA error checking

- Nsight profiling

Quantum GPU simulation

- GPU state-vector storage
- Single-qubit gate kernels
- Controlled-gate kernels
- Measurement probability reductions
- Batched circuits
- Memory bandwidth limits
- Kernel fusion
- CPU-GPU transfer overhead
- Multi-GPU conceptually

Updated textbooks and resources for this stage

Core textbooks

NVIDIA CUDA C++ Programming Guide — free

Use as your official CUDA reference.

CUDA by Example by Sanders and Kandrot — paid

Use as your beginner CUDA book. It is practical and example-heavy.

Programming Massively Parallel Processors by Hwu, Kirk, and El Hajj — paid

Use as your main serious GPU programming textbook. It teaches parallel patterns, memory coalescing, tiling, reductions, and GPU performance thinking.

Optional / advanced textbooks

Professional CUDA C Programming by Cheng, Grossman, and McKercher — paid

Use for practical CUDA programming and performance tuning.

CUDA Handbook by Nicholas Wilt — paid

Use as a more advanced CUDA reference.

Computer Architecture: A Quantitative Approach by Hennessy and Patterson — paid

Use if you want deeper hardware architecture knowledge.

Supporting resources

- NVIDIA Nsight Systems documentation
- NVIDIA Nsight Compute documentation
- NVIDIA CUDA samples
- cuQuantum documentation
- CUDA-Q documentation

Project 13: CUDA quantum gate kernels

Build:

cuda-statevector-kernels

Implement:

- State-vector allocation on GPU
- Initialize basis state
- Apply X gate
- Apply Z gate
- Apply H gate
- Apply arbitrary single-qubit 2x2 unitary
- Apply CNOT
- Compute norm
- Compare GPU output to CPU output
- Benchmark GPU vs OpenMP CPU

Important analysis:

For many state-vector gate kernels, performance is often memory-bandwidth limited. Your report should estimate bytes moved per amplitude update and compare against observed bandwidth.

Project 14: cuQuantum comparison project

Use cuQuantum or CUDA-Q examples and compare:

- Your CPU simulator
- Your OpenMP simulator
- Your CUDA kernels
- cuQuantum-backed simulator, when possible
- Qiskit Aer GPU, when possible

Move on when

You can:

- Write a basic CUDA kernel
 - Explain memory coalescing
 - Explain GPU occupancy at a high level
 - Use Nsight to inspect performance
 - Compare CPU vs GPU fairly
 - Explain when GPU acceleration helps and when transfer overhead dominates
 - Run a small quantum circuit simulation on GPU
-

Stage 8 — Months 21–24: advanced quantum simulation methods

Now you learn how real high-performance quantum simulators work.

Topics to learn

State-vector simulation

- Direct amplitude updates
- Qubit indexing
- Gate fusion
- Vectorization
- Memory layout
- Cache-aware ordering
- Distributed state vectors
- Communication-heavy gates

Density-matrix simulation

- Mixed states
- Noise channels
- Kraus operators
- Density matrix memory scaling
- Why density matrices are much more expensive than state vectors

Stabilizer simulation

- Clifford gates
- Stabilizer states
- Tableau representation
- Gottesman-Knill theorem conceptually
- Why stabilizer simulation can be efficient
- Limits of stabilizer methods

Tensor-network simulation

- Tensor contraction
- Matrix product states
- Entanglement and bond dimension
- Contraction ordering
- Circuit cutting conceptually
- When tensor networks beat state vectors
- When they fail

Quantum compiler/simulator systems topics

- Circuit IR
- Gate decomposition
- Circuit optimization

- Routing
- Mapping
- Hardware topology
- Noise-aware compilation
- Shot batching
- Benchmark design

Updated textbooks and resources for this stage

Core textbooks

Quantum Computation and Quantum Information by Nielsen and Chuang — paid
Use for density matrices, noise channels, quantum operations, stabilizers, and QEC basics.

Quantum Information Theory by Mark Wilde — free/paid depending on format
Use for quantum channels, entropy, noise models, and more advanced quantum information theory.

Introduction to Tensor Network Methods by Bridgeman and Chubb — free article/notes
Use as your entry point into tensor networks, matrix product states, contraction ordering, and simulation methods.

Numerical Linear Algebra by Trefethen and Bau — paid
Continue using for matrix decompositions, numerical stability, and linear algebra algorithms.

Matrix Computations by Golub and Van Loan — paid reference
Use as a deeper reference when implementing or studying numerical algorithms.

Optional textbooks

Classical and Quantum Computation by Kitaev, Shen, and Vyalov — paid
Use for more mathematical quantum computation, quantum complexity, and fault-tolerance foundations.

Tensor Network Methods in Many-body Physics / tensor-network review literature — optional
Use only after you have the basics. Tensor-network learning is often better done with review papers and coding projects than a single beginner textbook.

Supporting resources

Study real simulator projects:

- qsim
- QuEST
- Qulacs
- Qiskit Aer
- Cirq + qsim
- cuQuantum

- CUDA-Q
- UChicago Quantum Computer Systems lectures

Project 15: quantum simulator benchmark suite

Build:

quantum-simulator-benchmarks

Compare:

- Your Python simulator
- Your C++ simulator
- Your OpenMP simulator
- Your MPI simulator, if ready
- Your CUDA simulator, if ready
- Qiskit Aer
- qsim
- Qulacs
- QuEST, optional
- cuQuantum, optional

Circuit families:

- Bell/GHZ
- Random Clifford circuits
- Random universal circuits
- QFT
- Grover
- VQE-style ansatz
- QAOA-style ansatz

Metrics:

- Runtime
- Memory usage
- Gate throughput
- Max qubits simulated
- Scaling with depth
- Scaling with threads
- Scaling with MPI ranks
- Scaling with GPU

Your report should answer:

- Which simulator is fastest for which circuit?
- Which circuit families are memory-limited?

- Which are compute-limited?
- Which benefit from gate fusion?
- Which benefit from GPUs?
- Where does your simulator lose?
- What would you improve next?

Project 16: mini tensor-network simulator

Build a small tensor-network simulator in Python.

Minimum features:

- Represent each gate as a tensor
- Contract a small circuit
- Compare against state-vector output
- Experiment with different contraction orders
- Show when tensor methods use less memory
- Show when tensor methods become expensive

This project shows that you understand there is not just one way to simulate quantum circuits.

Move on when

You can:

- Explain state-vector, density-matrix, stabilizer, and tensor-network simulation
 - Explain why no simulator is best for every circuit
 - Benchmark at least three external simulators
 - Explain gate fusion
 - Explain qubit ordering and memory layout
 - Produce fair, reproducible benchmark plots
 - Read C++ simulator source code without being completely lost
-

Stage 9 — Months 24-27: quantum error correction and high-performance decoding

This is one of the best specializations for Quantum Computing + HPC roles.

Fault-tolerant quantum computing requires massive classical processing:

- Syndrome extraction
- Decoding
- Scheduling
- Feedback

- Simulation
- Benchmarking
- Logical error-rate estimation
- Real-time classical processing

Topics to learn

Quantum error correction basics

- Bit-flip code
- Phase-flip code
- Shor code conceptually
- Stabilizer formalism
- Pauli group
- Clifford gates
- Syndrome measurement
- Logical qubits
- Code distance
- CSS codes
- Surface code
- Repetition code
- Logical error rate
- Threshold concept
- Circuit-level noise

Decoding

- Syndrome graph
- Minimum-weight perfect matching
- Union-find decoding conceptually
- Belief propagation conceptually
- Sparse graph decoding
- Real-time decoding constraints
- Monte Carlo sampling
- Logical error-rate estimation
- Parallel decoding benchmarks

HPC connection

- Large-scale Monte Carlo
- Many independent shots
- Job arrays
- Parallel sampling
- Distributed parameter sweeps
- Decoder latency
- Throughput

- Memory layout for syndrome data
- Bit-packing
- SIMD opportunities
- GPU opportunities

Updated textbooks and resources for this stage

Core textbooks / notes

Surviving as a Quantum Computer in a Classical World by Daniel Gottesman — free draft
Use as a major QEC and fault-tolerance text.

Quantum Error Correction: An Introductory Guide by Joschka Roffe — free
Use as your first QEC guide. Good for detection/correction codes, surface code, and implementation issues.

Quantum Computation and Quantum Information by Nielsen and Chuang — paid
Use for QEC basics, stabilizers, noise, and quantum operations.

Preskill lecture notes — free
Use for deeper quantum information, stabilizer codes, and fault-tolerance theory.

Philippe Faist Quantum Error Correction notes — free
Use for modern QEC course-style notes.

Paid / specialized QEC references

Quantum Error Correction edited by Lidar and Brun — paid
Use as an advanced QEC reference, not as your first QEC book.

Classical and Quantum Computation by Kitaev, Shen, and Vyalı — paid
Use for more theoretical foundations.

Classical algorithms support

Open Data Structures — free
Use for data structures.

Introduction to Algorithms by CLRS — paid
Use for graph algorithms, matching-related background, complexity, and algorithm design.

Algorithms by Dasgupta, Papadimitriou, and Vazirani — free
Use as a readable algorithms supplement.

Supporting tools/resources

- Stim
- PyMatching
- Sinter
- Error Correction Zoo
- Surface-code tutorials

- QEC papers and documentation

Project 17: repetition-code decoder benchmark

Start simple.

Build:

repetition-code-decoder

Features:

- Simulate bit-flip noise
- Generate syndromes
- Decode using majority vote
- Decode using graph matching
- Estimate logical error rate
- Sweep physical error rate
- Sweep code distance
- Plot logical error rate vs physical error rate

Deliverables:

- Clean code
- Tests
- Plots
- Explanation of syndrome measurement
- Explanation of code distance
- Explanation of threshold behavior conceptually

Project 18: surface-code Monte Carlo benchmark

Build:

surface-code-decoding-benchmark

Use:

- Stim
- PyMatching
- Sinter
- Python multiprocessing
- Optional Slurm job arrays

Features:

- Generate surface-code circuits
- Sample detection events

- Decode with PyMatching
- Estimate logical error rates
- Sweep code distance
- Sweep physical error rate
- Benchmark decoder throughput
- Benchmark parallel scaling

Project 19: write your own small decoder

After using PyMatching, implement a toy decoder yourself.

Options:

- Minimum-distance decoder for tiny repetition code
- Union-find-style decoder for repetition code
- Brute-force decoder for small stabilizer code
- Belief propagation toy decoder for simple parity-check matrices

Do not try to beat PyMatching. The point is to understand the mechanics.

Move on when

You can:

- Explain what a stabilizer is
 - Explain syndrome measurement
 - Explain code distance
 - Simulate a repetition code
 - Simulate a surface-code experiment with Stim
 - Decode with PyMatching
 - Produce logical error-rate plots
 - Benchmark decoder throughput
 - Explain why decoding is an HPC/real-time computing problem
-

Stage 10 — Months 27–30: flagship capstone and open-source readiness

At this point, you need one **flagship project** that makes your résumé memorable.

Updated textbooks and resources for this stage

Core textbooks

The Pragmatic Programmer by Andrew Hunt and David Thomas — paid

Use for professional coding habits, project organization, debugging, and software craftsmanship.

Software Engineering at Google — free online / paid print

Use for maintainability, code review, testing, large-scale software engineering, and engineering culture.

Writing Science by Joshua Schimel — paid

Use to make your capstone report clear, persuasive, and readable.

Technical references

At this stage, source code and documentation become as important as textbooks. Study:

- [qsim source and documentation](#)
- [QuEST source and documentation](#)
- [Qiskit Aer documentation](#)
- [Stim documentation](#)
- [PyMatching documentation](#)
- [cuQuantum examples](#)
- [CUDA-Q examples](#)
- [OpenMP/MPI documentation](#)
- [Slurm documentation](#)

Capstone option A: distributed quantum circuit simulator

Best for:

- Quantum simulator engineer
- Quantum-HPC software engineer
- Scientific computing engineer
- GPU/distributed systems roles

Build:

distributed-quantum-simulator

Features:

- C++ core
- CMake
- Tests
- OpenMP backend

- MPI backend
- Optional CUDA backend
- Random circuit generator
- QFT/Grover/GHZ benchmarks
- Strong and weak scaling plots
- Comparison against qsim/Qulacs/Aer/QuEST
- Technical report

Stretch features:

- Gate fusion
- SIMD intrinsics
- GPU kernels
- Multi-node benchmark
- Python bindings
- Basic circuit parser

Résumé bullet:

Built a C++/OpenMP/MPI state-vector quantum circuit simulator with benchmarking against Qiskit Aer, qsim, and Qulacs; implemented gate kernels, measurement, random circuit generation, scaling tests, and reproducible performance reports.

Capstone option B: quantum error-correction decoding benchmark

Best for:

- QEC software engineer
- Quantum fault-tolerance roles
- Google Quantum AI-style decoding roles
- Research software roles

Build:

qec-decoder-benchmark-suite

Features:

- Stim-generated circuits
- PyMatching decoder integration
- Your toy decoder
- Sinter-based Monte Carlo
- Slurm job-array support
- Logical error-rate plots
- Decoder throughput benchmarks

- Scaling with code distance
- Scaling with number of CPU cores
- Report on latency and throughput

Stretch features:

- C++ decoder prototype
- Bit-packed syndrome data
- Parallel batch decoder
- GPU experiment
- Comparison with multiple decoders

Résumé bullet:

Built a quantum error-correction benchmark suite using Stim, PyMatching, and parallel Monte Carlo sampling to estimate logical error rates for repetition and surface-code circuits; added scaling benchmarks and Slurm-compatible parameter sweeps.

Capstone option C: GPU-accelerated quantum simulator

Best for:

- NVIDIA-style quantum software roles
- GPU simulation roles
- cuQuantum/CUDA-Q-adjacent work
- HPC software engineering

Build:

gpu-quantum-simulator

Features:

- CUDA state-vector backend
- Single-qubit gate kernels
- CNOT kernels
- Measurement probability reductions
- CPU reference implementation
- GPU correctness tests
- Nsight profiling
- Benchmark against CPU/OpenMP
- Compare against cuQuantum or Qiskit Aer GPU if possible

Stretch features:

- Gate fusion

- Batched circuits
- Mixed precision experiments
- Multi-GPU concept exploration
- CUDA-Q experiment

Résumé bullet:

Implemented CUDA kernels for state-vector quantum circuit simulation, including single-qubit gates, controlled gates, and measurement reductions; profiled with Nsight and benchmarked CPU vs GPU performance across circuit depths and qubit counts.

Capstone option D: quantum-centric supercomputing workflow prototype

Best for:

- IBM quantum-centric supercomputing roles
- Runtime/workflow roles
- HPC systems roles
- Hybrid quantum-classical infrastructure

Build:

quantum-hpc-workflow-orchestrator

Features:

- Python workflow layer
- Slurm scripts
- Parameter sweep manager
- Circuit generation
- Simulator backend selection
- Result collection
- Checkpointing
- Failed-job recovery
- Plot generation
- Reproducibility logs

Backends:

- Qiskit Aer
- qsim
- Qulacs
- Stim/PyMatching
- Your simulator

Résumé bullet:

Built a Slurm-compatible workflow system for hybrid quantum-HPC experiments, supporting simulator backend selection, parameter sweeps, checkpointing, result aggregation, and reproducible benchmark reports.

Full topic checklist

Use this as your master checklist.

Math checklist

You need:

- Algebra
- Trigonometry
- Complex numbers
- Euler's formula
- Vectors
- Matrices
- Inner products
- Norms
- Tensor products
- Eigenvalues/eigenvectors
- Hermitian matrices
- Unitary matrices
- Projection matrices
- Probability
- Random variables
- Expectation and variance
- Discrete math
- Graph theory
- Modular arithmetic
- Big-O notation
- Numerical linear algebra
- Sparse matrices
- Iterative methods
- FFT
- Optimization basics
- Floating-point arithmetic
- Numerical stability

Programming checklist

You need:

- Python
- NumPy
- SciPy
- Matplotlib
- pytest
- Git/GitHub
- Linux shell
- Jupyter
- C
- Modern C++
- CMake
- Debugging
- Profiling
- Unit testing
- Benchmarking
- CI basics
- Documentation
- Package environments
- Reproducible scripts

Quantum computing checklist

You need:

- Qubits
- Measurement
- Born rule
- State vectors
- Tensor products
- Single-qubit gates
- Two-qubit gates
- CNOT
- CZ
- SWAP
- Bell states
- GHZ states
- Entanglement
- No-cloning
- Teleportation

- Deutsch-Jozsa
- Bernstein-Vazirani
- Grover
- QFT
- Phase estimation
- Shor conceptually
- Hamiltonian simulation basics
- VQE and QAOA conceptually
- Noise channels
- Density matrices
- Stabilizers
- Clifford circuits
- QEC basics
- Surface code
- Syndrome decoding

HPC checklist

You need:

- Caches
- Memory bandwidth
- SIMD/vectorization
- Compiler optimization
- Profiling
- Roofline model
- OpenMP
- MPI
- CUDA
- Slurm
- Job arrays
- Strong scaling
- Weak scaling
- Amdahl's law
- Load balancing
- Race conditions
- False sharing
- Reductions
- Communication overhead
- GPU memory hierarchy
- Coalesced memory access
- Nsight profiling

- Reproducible benchmarking

Quantum-HPC specialization checklist

You need:

- State-vector simulation
 - Density-matrix simulation
 - Stabilizer simulation
 - Tensor-network simulation
 - Gate fusion
 - Qubit indexing
 - Qubit reordering
 - Circuit batching
 - Noise simulation
 - Random circuit benchmarks
 - Distributed state-vector partitioning
 - MPI communication patterns
 - GPU gate kernels
 - Tensor contraction ordering
 - QEC Monte Carlo sampling
 - Syndrome decoding
 - Logical error-rate estimation
 - Decoder throughput benchmarking
 - Quantum workflow orchestration
-

Best textbooks by category

Beginner programming

- **Think Python** — free
- **Dive Into Systems** — free
- **C Programming: A Modern Approach** — paid
- **A Tour of C++** — paid
- **Programming: Principles and Practice Using C++** — paid

Data structures and algorithms

- **Open Data Structures** — free
- **Algorithms** by Dasgupta, Papadimitriou, and Vazirani — free
- **Introduction to Algorithms** by CLRS — paid
- **Mathematics for Computer Science** by Lehman, Leighton, and Meyer — free

Linear algebra, probability, numerical methods

- **Introduction to Linear Algebra** by Gilbert Strang — paid
- **Linear Algebra Done Right** by Sheldon Axler — paid
- **Introduction to Probability** by Blitzstein and Hwang — paid
- **Numerical Linear Algebra** by Trefethen and Bau — paid
- **Scientific Computing: An Introductory Survey** by Michael T. Heath — paid
- **Matrix Computations** by Golub and Van Loan — paid reference
- **Scientific Python Lectures** — free

Quantum computing

- **Quantum Computing for the Quantum Curious** — free
- **Quantum Computer Science: An Introduction** by N. David Mermin — paid
- **Quantum Computation and Quantum Information** by Nielsen and Chuang — paid
- **Quantum Computing: Lecture Notes** by Ronald de Wolf — free
- **Introduction to Classical and Quantum Computing** by Thomas Wong — free
- **Quantum Computing: A Gentle Introduction** by Rieffel and Polak — paid
- **Dancing with Qubits** by Robert S. Sutor — paid

Systems and performance

- **Dive Into Systems** — free
- **Computer Systems: A Programmer's Perspective** — paid
- **Effective Modern C++** — paid
- **Optimized C++** — paid
- **Computer Architecture: A Quantitative Approach** — paid

HPC and parallel computing

- **The Art of HPC** — free
- **An Introduction to Parallel Programming** by Peter Pacheco — paid
- **Parallel Programming in C with MPI and OpenMP** by Michael J. Quinn — paid
- **Introduction to High Performance Computing for Scientists and Engineers** by Hager and Wellein — paid
- **Using MPI** — paid
- **Using OpenMP** — paid

GPU programming

- **NVIDIA CUDA C++ Programming Guide** — free
- **CUDA by Example** — paid
- **Programming Massively Parallel Processors** — paid
- **Professional CUDA C Programming** — paid
- **CUDA Handbook** — paid

Advanced quantum simulation

- **Quantum Computation and Quantum Information** — paid
- **Quantum Information Theory** by Mark Wilde — free/paid
- **Introduction to Tensor Network Methods** by Bridgeman and Chubb — free
- **Classical and Quantum Computation** by Kitaev, Shen, and Vyalov — paid
- **Numerical Linear Algebra** by Trefethen and Bau — paid
- **Matrix Computations** by Golub and Van Loan — paid reference

Quantum error correction

- **Surviving as a Quantum Computer in a Classical World** by Daniel Gottesman — free draft
- **Quantum Error Correction: An Introductory Guide** by Joschka Roffe — free
- **Quantum Computation and Quantum Information** by Nielsen and Chuang — paid
- **Preskill lecture notes** — free
- **Philippe Faist Quantum Error Correction notes** — free
- **Quantum Error Correction** edited by Lidar and Brun — paid reference

Professional polish

- **The Pragmatic Programmer** — paid
 - **Software Engineering at Google** — free online / paid print
 - **Writing Science** — paid
-

First 30 days: exact plan

Week 1

Study:

- CS50 Week 0/1
- Complex numbers basics
- Basic terminal and Git

Textbooks:

- **Think Python**, beginner chapters
- **Dive Into Systems**, early computing/C overview
- **Introduction to Linear Algebra** by Strang, vectors intro

Build:

- GitHub learning log
- Python script for complex numbers
- README explaining $a + bi$

Deliverable:

- `complex_numbers.py`
 - `notes/week_01.md`
-

Week 2

Study:

- Python functions
- Lists
- Loops
- Matrix basics
- 3Blue1Brown linear algebra chapters 1–4

Textbooks:

- **Think Python**, functions and lists
- **Strang**, matrix/vector basics
- **Open Data Structures**, arrays/lists intro

Build:

- Vector operations
- Matrix-vector multiplication
- Unit tests

Deliverable:

- `linear_algebra.py`
 - `test_linear_algebra.py`
-

Week 3

Study:

- Norms
- Inner products
- Complex conjugates
- Matrix multiplication
- MIT 18.06 early lectures

Textbooks:

- **Strang**, inner products and matrix multiplication
- **Introduction to Probability** by Blitzstein and Hwang, optional early probability

- **Dive Into Systems**, basic C/memory reading if ready

Build:

- Complex inner product
- Conjugate transpose
- Matrix multiplication
- Unitary-checking function

Deliverable:

- Notebook showing unitary matrices preserve norm
-

Week 4

Study:

- Qubit notation
- Measurement probabilities
- X, Y, Z, H gates

Textbooks:

- **Quantum Computing for the Quantum Curious**, first chapters
- **Quantum Computer Science** by Mermin, early qubit/gate sections
- **Strang**, continue linear algebra support

Build:

- Qubit class or simple functions
- Single-qubit simulator
- Measurement sampler
- Plot measurement frequencies

Deliverable:

- First quantum mini-project README

At the end of month 1, you should already have a small public repo.

What your GitHub should look like after each checkpoint

After 3 months

You should have:

- complex-linear-algebra-lab
- qubit-probability-calculator
- Learning log
- Tests
- Simple plots
- Clean READMEs

After 6 months

You should have:

- numpy-quantum-simulator
- Dense vs sparse Hamiltonian notebooks
- Runtime scaling plots
- Qubit memory scaling table

After 9 months

You should have:

- Quantum algorithms from scratch
- Qiskit/Cirq/PennyLane comparison
- Grover/QFT/teleportation demos
- Correctness tests against known results

After 12 months

You should have:

- Classical algorithm baselines
- FFT vs QFT project
- Sparse numerical linear algebra demos
- Cleaner code quality

After 15 months

You should have:

- C++ state-vector simulator
- CMake build
- Unit tests
- Benchmark report
- Performance optimization write-up

After 18 months

You should have:

- OpenMP simulator
- MPI simulator prototype
- Slurm scripts
- Strong/weak scaling plots

After 21 months

You should have:

- CUDA quantum gate kernels
- GPU vs CPU benchmark
- Nsight profiling notes
- Optional cuQuantum comparison

After 24 months

You should have:

- Simulator benchmark suite
- Tensor-network mini-simulator
- Comparison against qsim/Qulacs/Aer/QuEST

After 27–30 months

You should have:

- QEC decoder benchmark
 - Surface-code logical error plots
 - Flagship capstone
 - Technical report
 - Blog post or demo video
 - Open-source contribution attempt
-

Resume-worthy project ranking

If you only have time to polish six projects, polish these:

1. C++/OpenMP/MPI quantum simulator

Most important for Quantum-HPC roles.

Shows:

- C++
- Quantum simulation
- Parallel computing

- Memory scaling
- Benchmarking

2. CUDA quantum gate kernels

Shows:

- GPU programming
- Low-level performance
- Profiling
- Hardware awareness

3. Quantum simulator benchmark suite

Shows:

- Fair benchmarking
- Knowledge of real tools
- Engineering maturity

4. Surface-code decoder benchmark

Shows:

- QEC knowledge
- Parallel Monte Carlo
- Stim/PyMatching
- Fault-tolerance relevance

5. QFT / FFT / phase-estimation project

Shows:

- Quantum algorithms
- Classical numerical comparison
- Mathematical maturity

6. Tensor-network mini-simulator

Shows:

- Advanced simulation awareness
 - Ability to go beyond state vectors
 - Understanding of circuit structure
-

Weekly schedule

A good weekly rhythm:

Time	Activity
3 hours	Math / theory
3 hours	Programming practice
3 hours	Project work
2 hours	Reading documentation / papers / notes
1 hour	Writing README, notes, or benchmark report

Every week should produce something visible:

- A solved problem set
- A notebook
- A test
- A benchmark
- A plot
- A README section
- A short explanation
- A small pull request
- A performance report

Watching videos without coding does not count as progress.

How to know you are becoming competitive

You are **beginner-ready for research assistant roles** when you can:

- Use Python and Git comfortably
- Explain qubits and gates
- Build small circuits
- Implement Bell/Grover/QFT
- Use Qiskit/Cirq/PennyLane
- Write clear READMEs

You are **HPC-ready for campus research computing roles** when you can:

- Write C++ code
- Use CMake
- Use OpenMP

- Run MPI examples
- Use Slurm scripts
- Produce scaling plots
- Explain performance bottlenecks

You are **quantum-HPC internship-ready** when you can:

- Build a C++ state-vector simulator
- Parallelize it
- Benchmark it
- Compare against real simulators
- Explain memory scaling
- Use at least one GPU or MPI workflow
- Discuss QEC decoding at a basic level

You are **strong for top quantum-HPC roles** when you can:

- Explain state-vector, stabilizer, density-matrix, and tensor-network simulation
 - Use Qiskit Aer/qsim/Qulacs/QuEST/cuQuantum intelligently
 - Build or modify performance-critical simulator code
 - Run reproducible benchmarks
 - Use OpenMP/MPI/CUDA
 - Build a QEC decoding benchmark
 - Discuss decoder throughput, logical error rate, and scaling
-

What not to waste time on

Avoid these traps:

- Only watching quantum explainer videos
- Only using drag-and-drop circuit tools
- Only doing Qiskit notebooks without understanding the math
- Claiming quantum advantage in beginner projects
- Ignoring C++ because Python feels easier
- Ignoring classical algorithms
- Ignoring benchmarking
- Ignoring failure cases
- Over-focusing on quantum machine learning too early
- Building many tiny projects but polishing none

For Quantum Computing + HPC, your strongest signal is:

“I can build fast, correct, tested, benchmarked software for quantum workloads.”

That is much stronger than saying:

“I completed a quantum computing course.”

Final target portfolio

By the end, aim for this GitHub pinned-project list:

1. [cpp-statevector-simulator](#)

C++ quantum simulator with tests and benchmarks.

2. [parallel-quantum-simulator](#)

OpenMP/MPI version with strong/weak scaling.

3. [cuda-quantum-kernels](#)

CUDA gate kernels with Nsight profiling.

4. [quantum-simulator-benchmarks](#)

Benchmarks against Qiskit Aer, qsim, Qulacs, and QuEST.

5. [surface-code-decoding-benchmark](#)

Stim/PyMatching/Sinter QEC benchmark with logical error plots.

6. [tensor-network-mini-simulator](#)

Small tensor-network simulator showing contraction-order tradeoffs.

7. [quantum-algorithms-from-scratch](#)

Grover, QFT, phase estimation, teleportation, and classical baselines.

That portfolio would make you look far more serious than most undergraduates casually interested in quantum computing.